RNN

1 순서가 있는 자료

2 RNN 의 구조와 식

3 RNN 을 직접 계산하기

4 RNN 의 입출력 형태와 쓰임

5 RNN 의 학습과 한계

1

순서가 있는 자료

순서가 뜻을 가진 자료에는 왜 MLP 와 다른 구조가 필요한가?

표 자료와 시계열 자료, 가게 손님 수 28일, MLP 와 RNN 이 입력을 받는 방식

표 자료는 행의 순서를 바꿔도 뜻이 같고, 시계열 자료는 행의 순서가 뜻을 가진다

두 표 모두 변수가 여럿인 **다변량 자료(multivariate data)**다. 다른 것은 행의 순서다.

자동차 5대의 표

	주행거리	마력	용량	가격
자동차 1	46,986	90	2,000	13,500
자동차 2	19,700	192	1,800	21,500
자동차 3	71,138	69	1,800	12,950
자동차 4	31,461	192	1,800	20,950
자동차 5	43,610	192	1,800	19,950

행의 순서를 바꿔도 같은 자료다

자동차 표는 행(관측)끼리 독립이라고 가정한다.

센서 다섯의 시각별 값

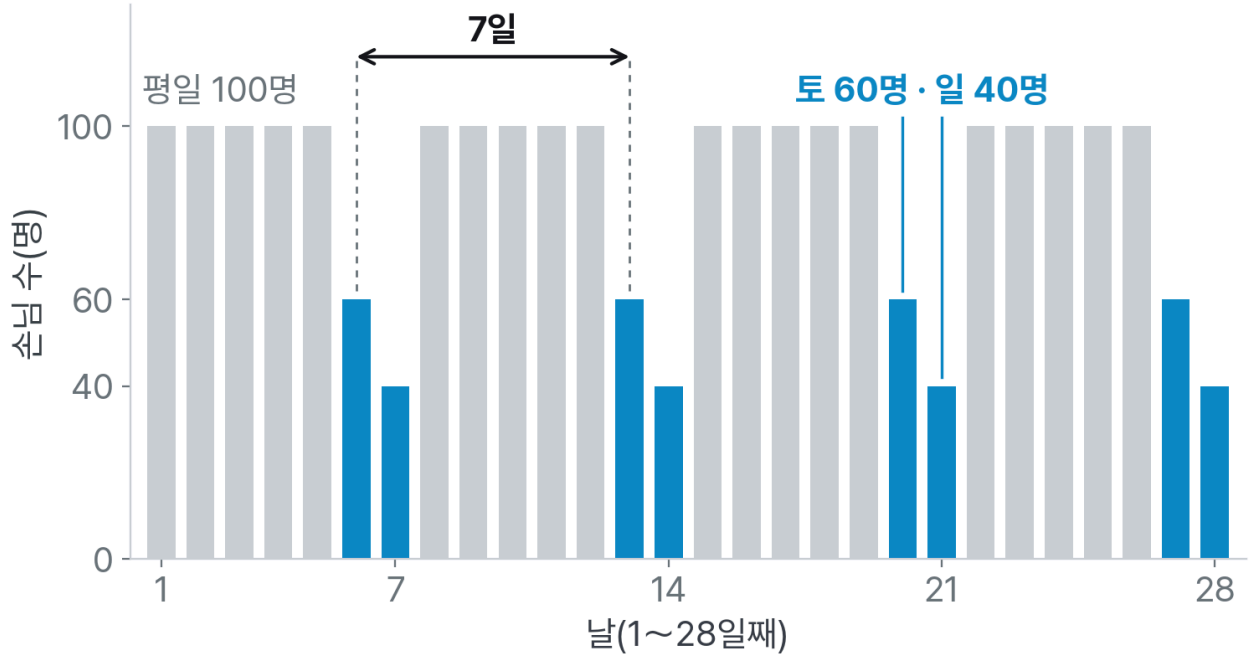
시간

시각	센서 1	센서 2	센서 3	센서 4	센서 5	상태
12:00	0	98.9	3.9	0.19	0.21	정상
13:00	0	98.9	3.9	0.21	0.23	정상
14:00	0.3	74.5	6.7	0.23	0.24	불량
15:00	6.8	98.9	7.9	0.21	0.22	불량
16:00	−0.9	78.3	6.6	0.21	0.23	정상
17:00	−1	79.2	6.6	0.23	0.21	불량

센서 표는 지금 값이 앞 시각의 값에 영향을 받는 **시계열 자료(time series data)**다. 변수가 여럿이라 다변량 시계열이다.

가게 손님 수는 평일 100명 · 토요일 60명 · 일요일 40명이 7일마다 반복된다

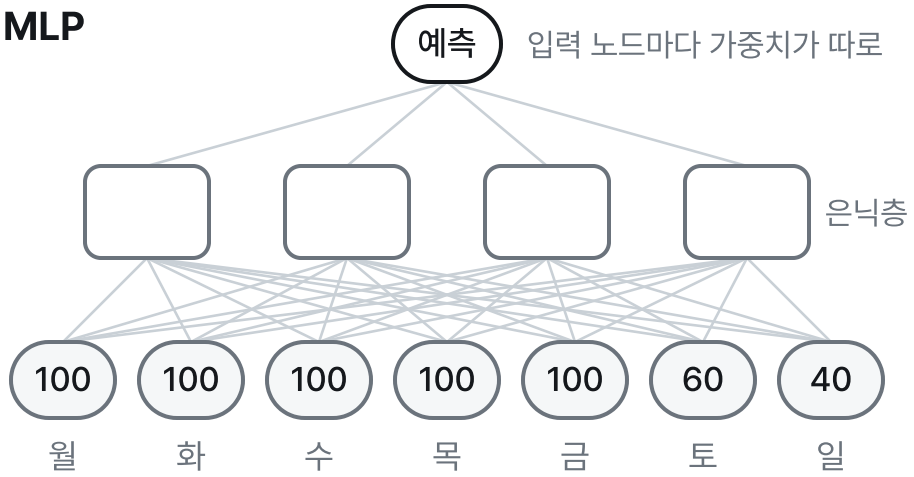
요일	월	화	수	목	금	토	일
손님 수(명)	100	100	100	100	100	**60**	**40**



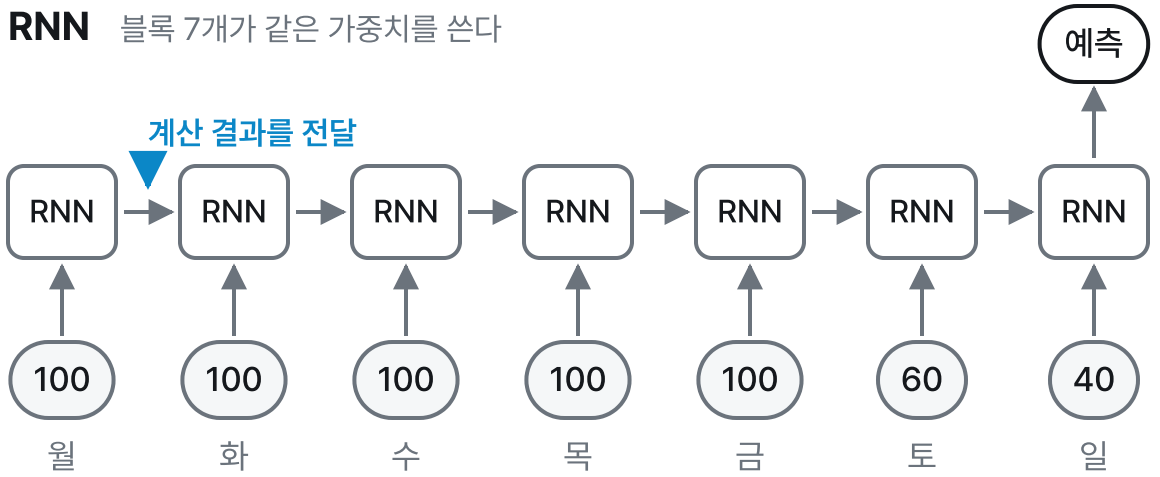
- RNN 의 계산 예와 다음 자료 「LSTM」의 계산 예에 이 가게의 손님 수를 입력으로 쓴다(딥러닝 시계열 2편과 같은 가게).
- 시계열 자료에는 추세 · 계절성 · 주기 같은 특징이 있다(시계열 과목).
- 이 가게는 **7일 계절성**이 있다. 같은 요일의 손님 수가 7일마다 반복된다.
- 주가 · 기온 · 웹 접속 수도 시계열 자료다.

MLP 는 정해진 길이의 입력을 한꺼번에 받고, RNN 은 하루씩 받아 계산 결과를 다음 날로 전달한다

MLP



RNN 블록 7개가 같은 가중치를 쓴다



지난 값을 입력 노드에 나란히 두면 MLP 도 쓸 수 있다(딥러닝 시계열 1편의 28일 입력 MLP). 다만 입력 길이가 고정이고, 날마다 가중치가 따로이며, 앞 날의 계산 결과를 다음 날 계산에 전달하는 경로가 없다.

순환 신경망(recurrent neural network, RNN)은 입력을 하루씩 차례로 받는다. 블록마다 계산한 결과를 다음 날 계산에 전달한다. 예측은 다음 날 손님 수다.

정리: 순서가 뜻을 가진 자료에는 앞 시점의 계산 결과를 쓴다

1 시계열 자료

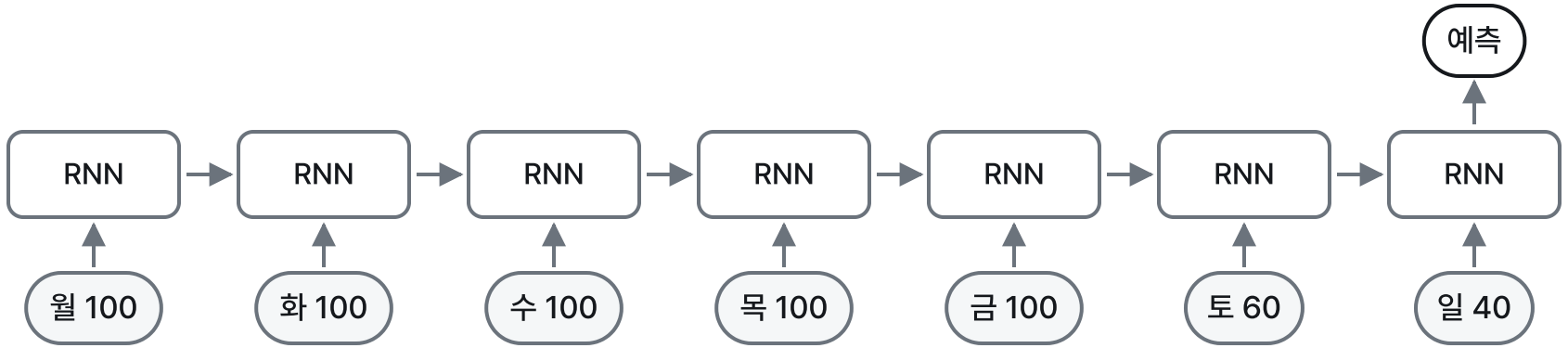
행의 순서가 뜻을 가진다. 지금 값이 앞 시각의 값에 영향을 받는다

2 MLP

입력 길이가 고정이고, 앞 날의 계산 결과를 전달하는 경로가 없다

3 RNN

입력을 하루씩 받아 계산 결과를 다음 날 계산에 전달한다



다음, 2. RNN 의 구조와 식: 한 시점의 계산과 전달을 그림과 식으로 본다.

2

RNN 의 구조와 식

RNN 은 한 시점에 무엇을 계산하고, 앞 시점과 어떻게 이어지나?

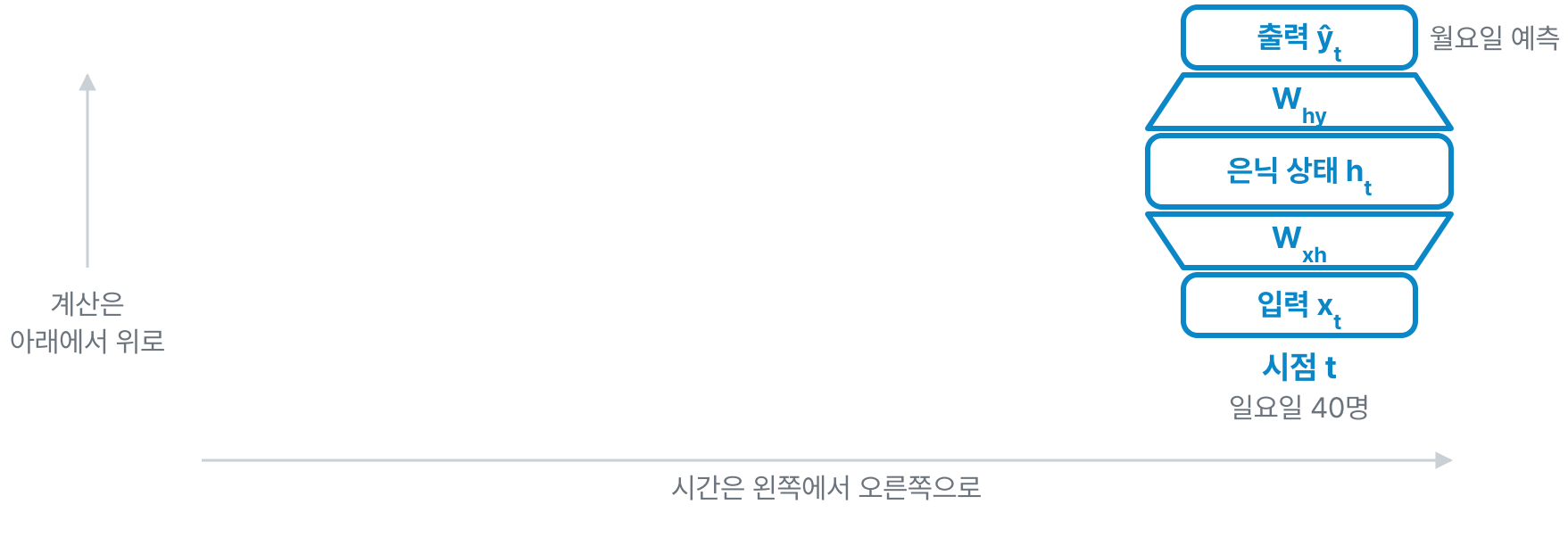
시점 t 의 계산에서 시작해 앞 시점을 하나씩 이어 그리고, 식 두 개로 정리한다

한 시점만 보면 RNN 은 입력 → 은닉 상태 → 출력을 계산하는 MLP 와 같다

「딥러닝 개요」의 MLP 를 아래에서 위로 그리고 층을 블록 하나로 묶었다. 은닉층 노드들의 값이 은닉 상태(hidden state) h_t 다. W_{xh} 는 입력 → 은닉, W_{hy} 는 은닉 → 출력의 가중치다.

$$h_t = \tanh(W_{xh} x_t)$$
$$\hat{y}_t = g(W_{hy} h_t)$$

g: 출력에 적용하는 함수(손님 수 예측에서는 그대로 둔다)



앞 시점 $t-1$ 에서도 같은 가중치 W_{xh} 로 은닉 상태를 계산한다

한 시점 전에도 입력 x_{t-1} 에 같은 W_{xh} 를 곱해 은닉 상태 h_{t-1} 을 계산했다.
출력은 필요한 시점에만 계산하므로 이 블록에는 출력을 그리지 않았다.

$$h_t = \tanh(W_{xh}x_t)$$
$$\hat{y}_t = g(W_{hy}h_t)$$

g: 출력에 적용하는 함수(손님 수 예측에서는 그대로 둔다)

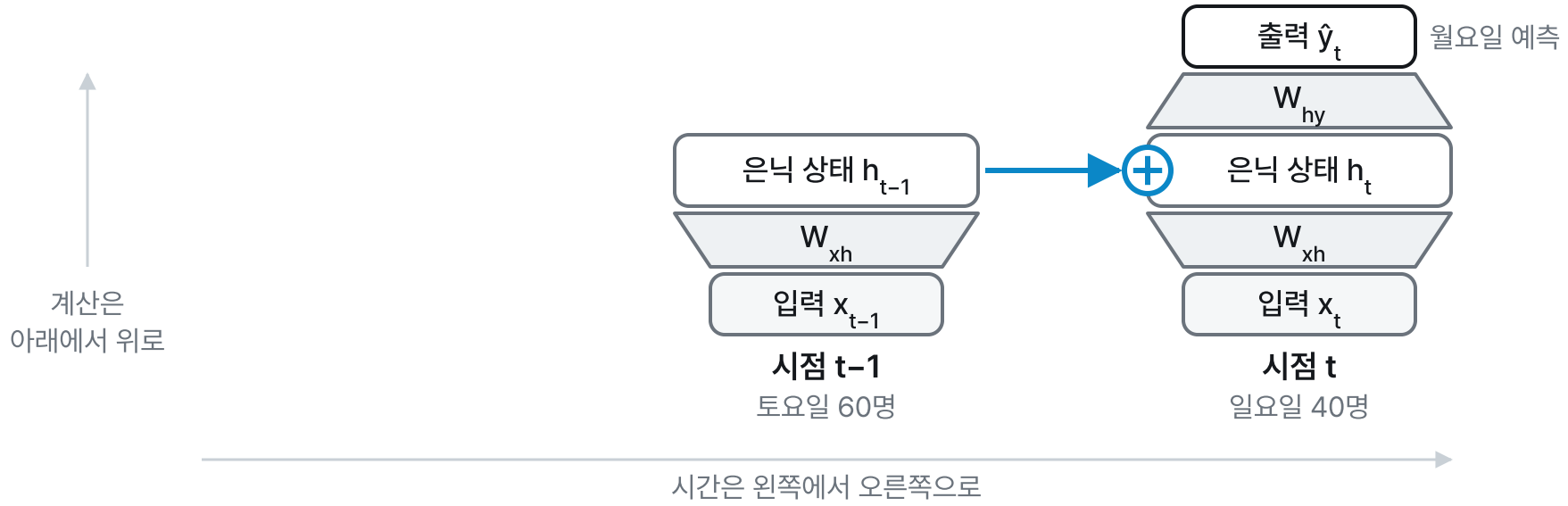


RNN 은 앞 시점의 은닉 상태를 이번 시점의 계산에 전달한다

RNN 의 목표는 앞 시점의 계산 결과를 반영해 더 나은 예측을 하는 것이다. 그래서 h_{t-1} 을 시점 t 의 은닉 상태 계산에 더한다. 그림의 ⊕ 는 덧셈이다. 오른쪽 식에는 전달할 때 곱하는 가중치의 이름을 정한 다음에 이 덧셈을 적는다.

$$h_t = \tanh(W_{xh}x_t)$$
$$\hat{y}_t = g(W_{hy}h_t)$$

g: 출력에 적용하는 함수(손님 수 예측에서는 그대로 둔다)

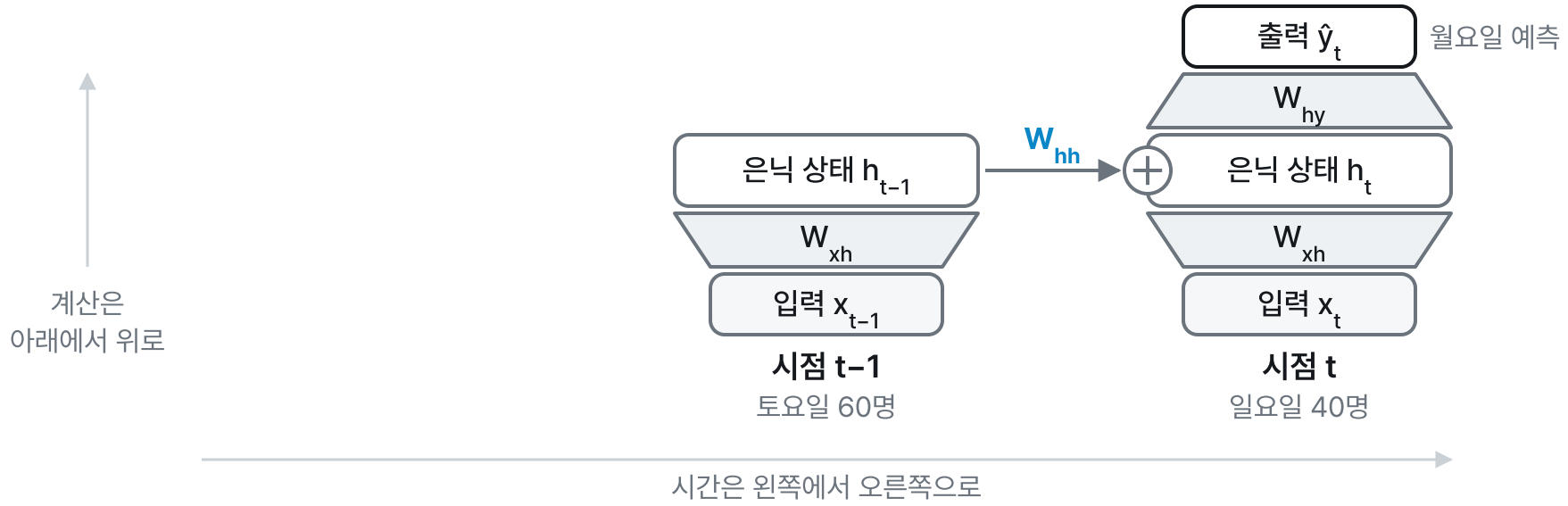


전달하는 은닉 상태에는 가중치 W_{hh} 를 곱한다

전달할 때도 가중치를 곱한다. 이 가중치의 이름이 W_{hh} 다. 첨자 hh 는 은닉 → 은닉, 곧 앞 시점의 은닉 상태를 이번 시점의 은닉 상태 계산에 쓴다는 뜻이다.

$$h_t = \tanh(W_{xh}x_t)$$
$$\hat{y}_t = g(W_{hy}h_t)$$

g: 출력에 적용하는 함수(손님 수 예측에서는 그대로 둔다)



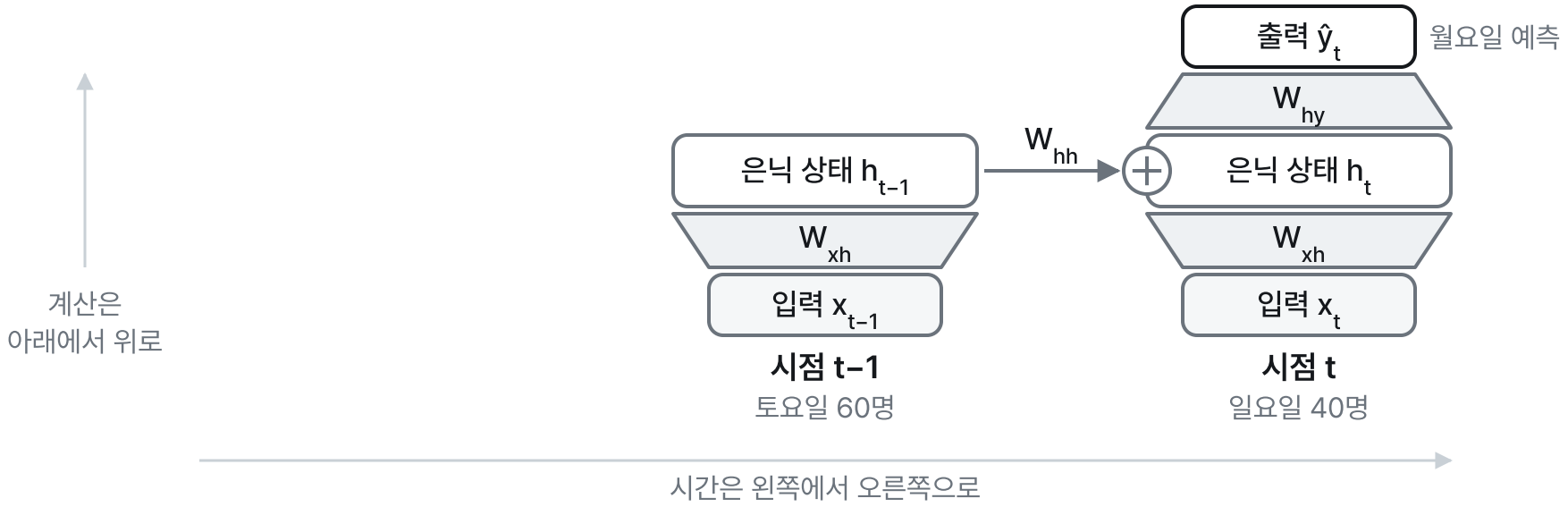
은닉 상태는 이번 입력과 앞 은닉 상태에 각각 가중치를 곱해 더한 뒤 tanh 를 적용한 값이다

그림의 가로 화살표가 식의 둘째 항이다. 앞 은닉 상태 h_{t-1} 에 W_{hh} 를 곱해 $W_{xh}x_t$ 에 더하고, 그 합에 tanh 를 적용한다.

W_{hh}: 앞 시점의 은닉 상태에 곱하는 가중치

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1})$$
$$\hat{y}_t = g(W_{hy}h_t)$$

g: 출력에 적용하는 함수(손님 수 예측에서는 그대로 둔다)

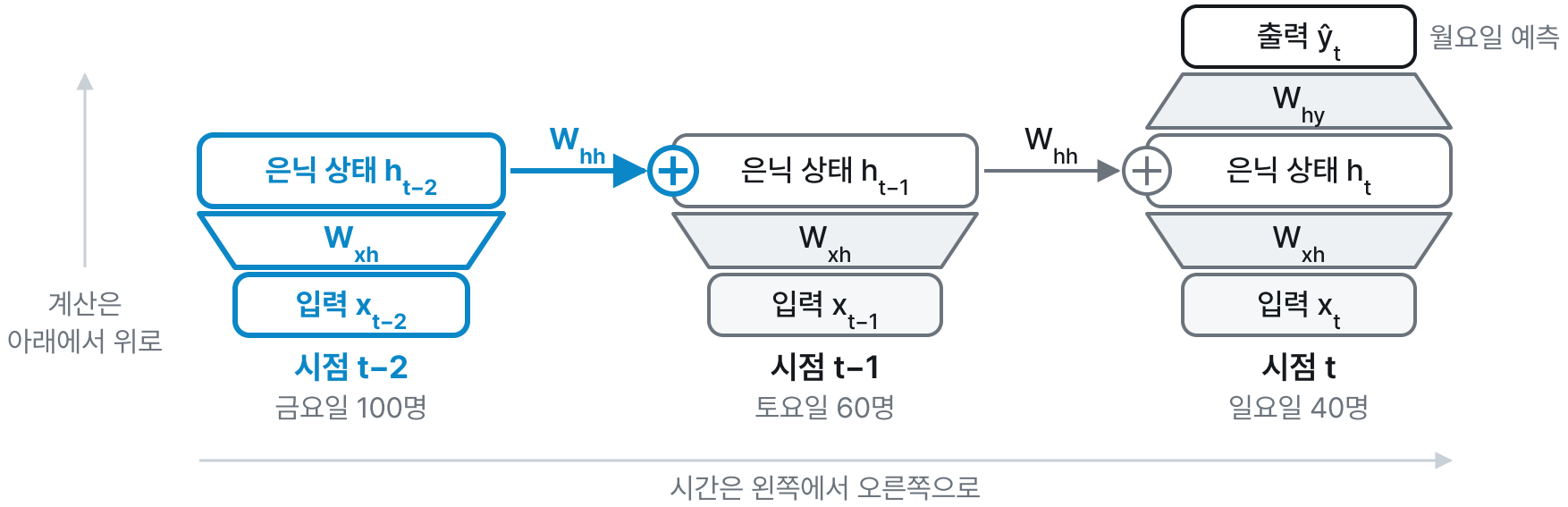


같은 계산을 시점마다 반복해서, 앞 은닉 상태 안에는 그 앞 시점의 값이 이미 반영되어 있다

한 시점 더 앞도 같은 계산이다. h_{t-1} 의 식 안에 h_{t-2} 가 있어 앞 시점들이 차례로 반영된다. 블록 셋의 W_{xh} 는 같은 값이고, 화살표 둘의 W_{hh} 도 같은 값이다.

$$h_{t-1} = \tanh(W_{xh}x_{t-1} + W_{hh}h_{t-2})$$
$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1})$$
$$\hat{y}_t = g(W_{hy}h_t)$$

g: 출력에 적용하는 함수(손님 수 예측에서는 그대로 둔다)

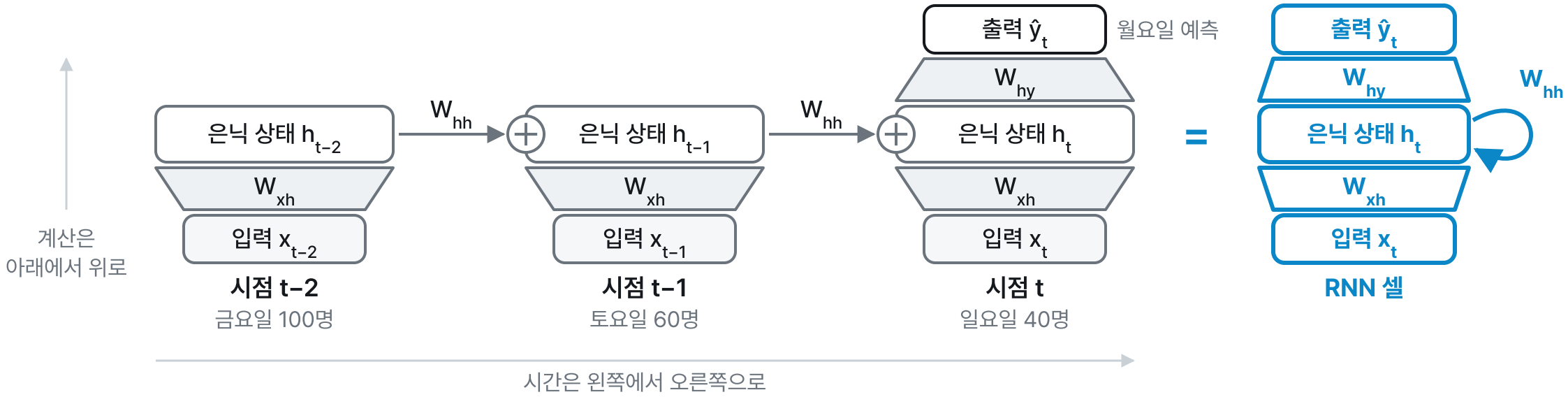


시점마다 그린 블록 셋은 블록 하나를 접어 그린 그림과 같다

블록 셋은 가중치 묶음 셋이 아니라 가중치 묶음 하나를 시점마다 다시 쓴 것이다. 그래서 고리 화살표 하나로 접어 그린다. 이 블록 하나를 RNN 셀(cell)이라 부른다. 입력 길이가 달라져도 같은 가중치로 계산한다.

$$h_{t-1} = \tanh(W_{xh}x_{t-1} + W_{hh}h_{t-2})$$
$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1})$$
$$\hat{y}_t = g(W_{hy}h_t)$$

g: 출력에 적용하는 함수(손님 수 예측에서는 그대로 둔다)



RNN 의 식은 두 개이고, 가중치 $W_{xh} \cdot W_{hh} \cdot W_{hy}$ 는 모든 시점에서 같은 값이다

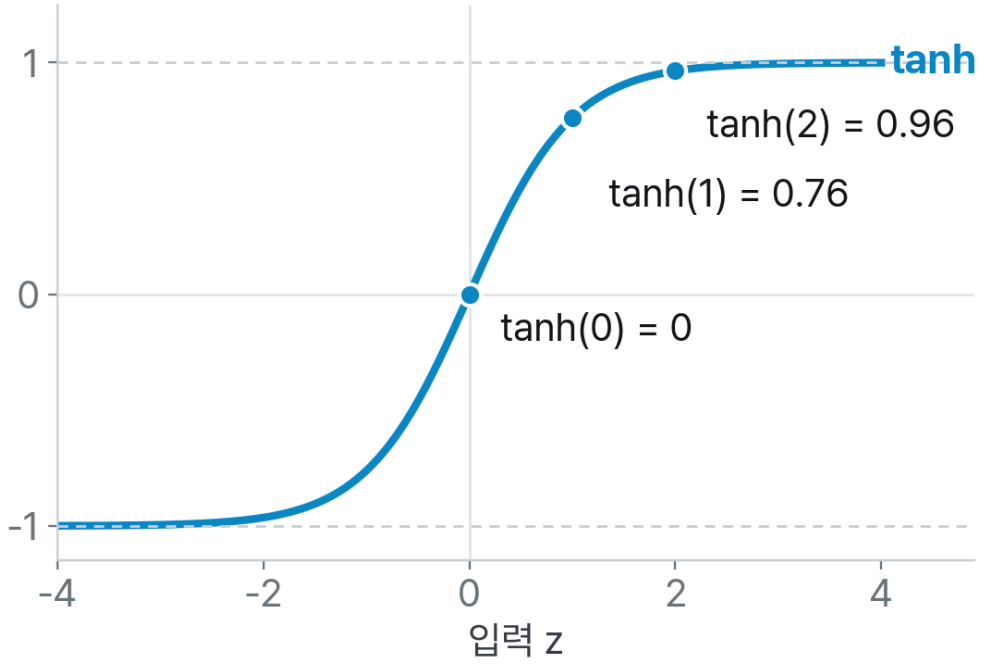
진행 그림의 식 둘에 편향을 더하면 RNN 의 식이다.

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$
$$\hat{y}_t = g(W_{hy}h_t + b_y)$$

$b_h \cdot b_y$: 편향. 진행 그림의 식과 다음 계산 예(가게의 나흘)에서는 편향을 0 으로 둔다

g: 분류면 소프트맥스, 수를 예측하면 그대로 · 첫 시점의 앞 은닉 상태 h_0 은 0 · 노드가 여럿이면 (센서 다섯 값처럼) 가중치도 수 여러 개다

기호	이름	하는 일
x_t	입력	시점 t 에 입력하는 값
h_t	은닉 상태	시점 t 까지의 계산 결과. 다음 시점에 전달한다
W_{xh}	입력 → 은닉 가중치	입력 x_t 에 곱한다
W_{hh}	은닉 → 은닉 가중치	앞 시점의 은닉 상태 h_{t-1} 에 곱한다
W_{hy}	은닉 → 출력 가중치	은닉 상태 h_t 에 곱해 출력 $\hat{y}_t$ 를 계산한다



어떤 수든 −1 과 1 사이로 바꾼다(「딥러닝 개요」의 활성화 함수)

정리: RNN 은 같은 가중치로 시점마다 은닉 상태를 새로 계산해 다음 시점에 전달한다

1 한 시점의 계산

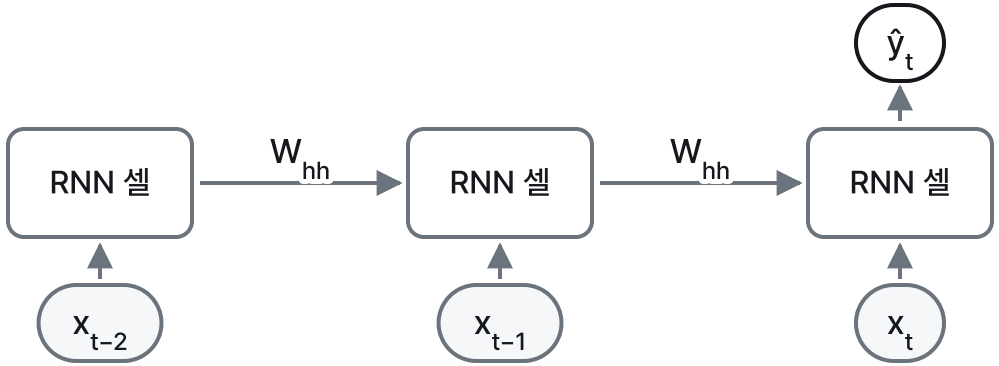
입력과 앞 은닉 상태에 각각 가중치를 곱해 더하고 tanh 를 적용한다

2 가중치 묶음은 하나

$W_{xh} \cdot W_{hh} \cdot W_{hy}$ 는 모든 시점에서 같은 값이다

3 은닉 상태를 전달

새로 계산한 은닉 상태를 다음 시점의 계산에 전달한다



다음, 3. RNN 을 직접 계산하기: 가게의 나흘을 입력해 은닉 상태를 수로 구한다.

3

RNN 을 직접 계산하기

가게의 나흘을 입력하면 은닉 상태는 날마다 얼마가 되나?

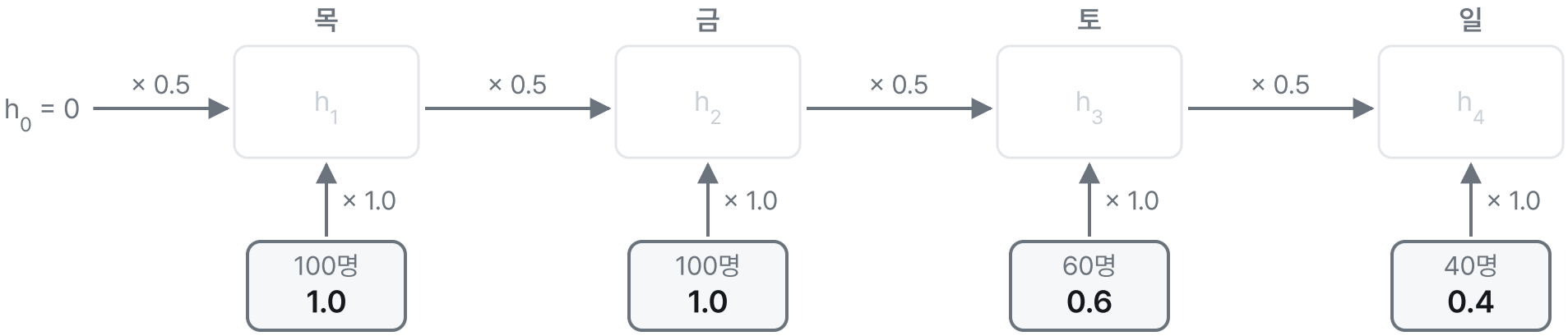
목 · 금 · 토 · 일 손님 수를 가중치 1.0 과 0.5 로 계산하고 PyTorch 출력과 대조한다

계산 예: 가게의 목 · 금 · 토 · 일 손님 수를 가중치 1.0 과 0.5 인 RNN 에 차례로 입력한다

간단한 예로 계산해 본다. tanh(100) 은 1 이라 손님 수를 그대로 입력하면 날마다 같은 값이 된다. 그래서 100명을 1.0 으로 줄인다. tanh 값은 파이썬 `math.tanh` 로 구한다.

$$h_t = \tanh(w_x \times x_t + w_h \times h_{t-1})$$

$w_x = 1.0 \cdot w_h = 0.5$ ($W_{xh} \cdot W_{hh}$ 가 수 하나일 때의 이름) $\cdot h_0 = 0$

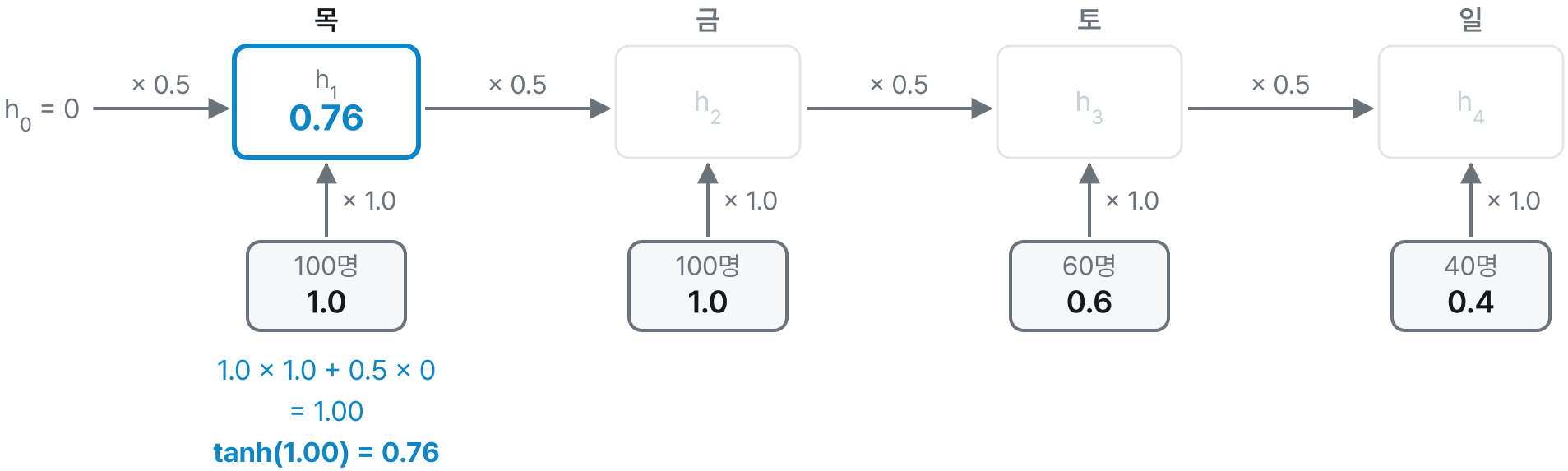


목요일: 앞 은닉 상태가 0 이어서 은닉 상태는 tanh(1.00) = 0.76 이다

첫날이라 앞 은닉 상태는 0 이다. 은닉 상태 0.76 은 손님 수가 아니라 다음 날 계산에 쓰는 중간값이다.

$$h_t = \tanh(w_x \times x_t + w_h \times h_{t-1})$$

$w_x = 1.0 \cdot w_h = 0.5$ ($W_{xh} \cdot W_{hh}$ 가 수 하나일 때의 이름) $\cdot h_0 = 0$

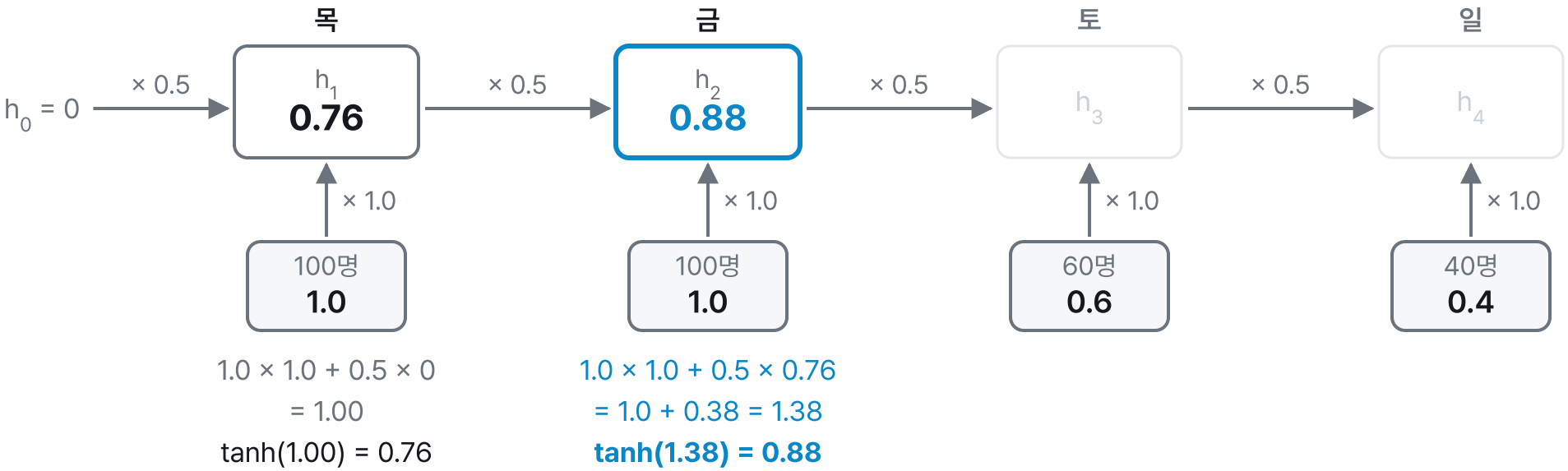


금요일: 입력 1.0 에 목요일 은닉 상태의 절반 0.38 을 더해 tanh(1.38) = 0.88 이 된다

입력 1.0 에 목요일 은닉 상태의 절반 0.5 × 0.76 = 0.38 을 더한다. 더한 값 1.38 에 tanh 를 적용한다.

$$h_t = \tanh(w_x \times x_t + w_h \times h_{t-1})$$

$w_x = 1.0 \cdot w_h = 0.5$ ($W_{xh} \cdot W_{hh}$ 가 수 하나일 때의 이름) $\cdot h_0 = 0$

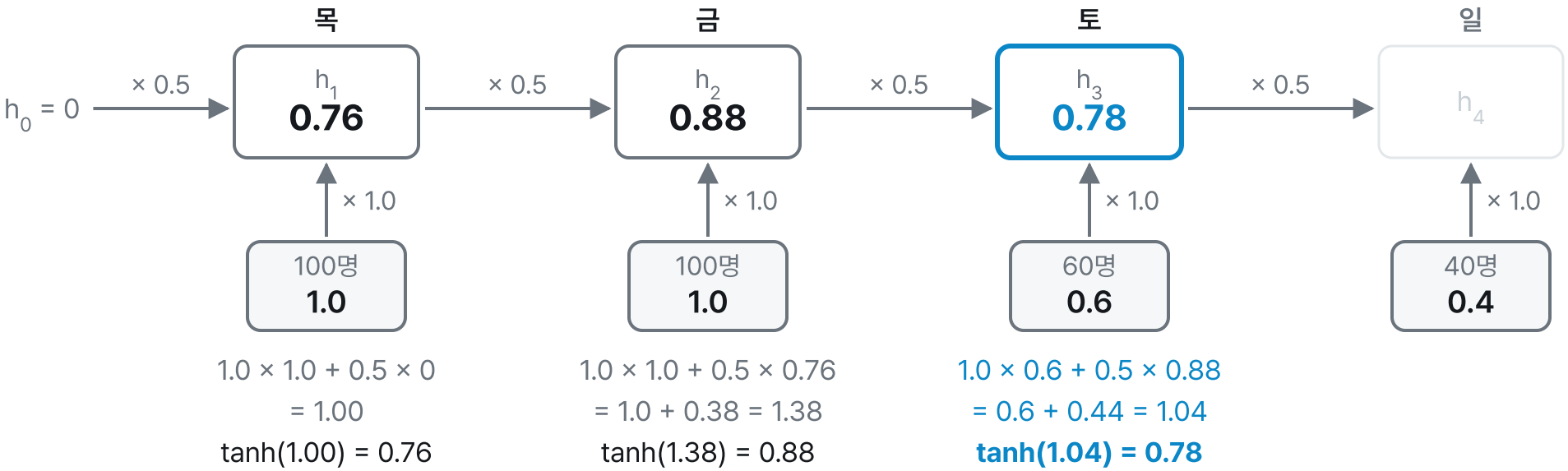


토요일: 1.0 × 0.6 + 0.5 × 0.88 = 1.04 이고 은닉 상태는 0.78 이다

입력은 0.6 으로 줄었지만 금요일 은닉 상태의 절반 0.44 가 더해진다.

$$h_t = \tanh(w_x \times x_t + w_h \times h_{t-1})$$

$w_x = 1.0 \cdot w_h = 0.5$ ($W_{xh} \cdot W_{hh}$ 가 수 하나일 때의 이름) $\cdot h_0 = 0$

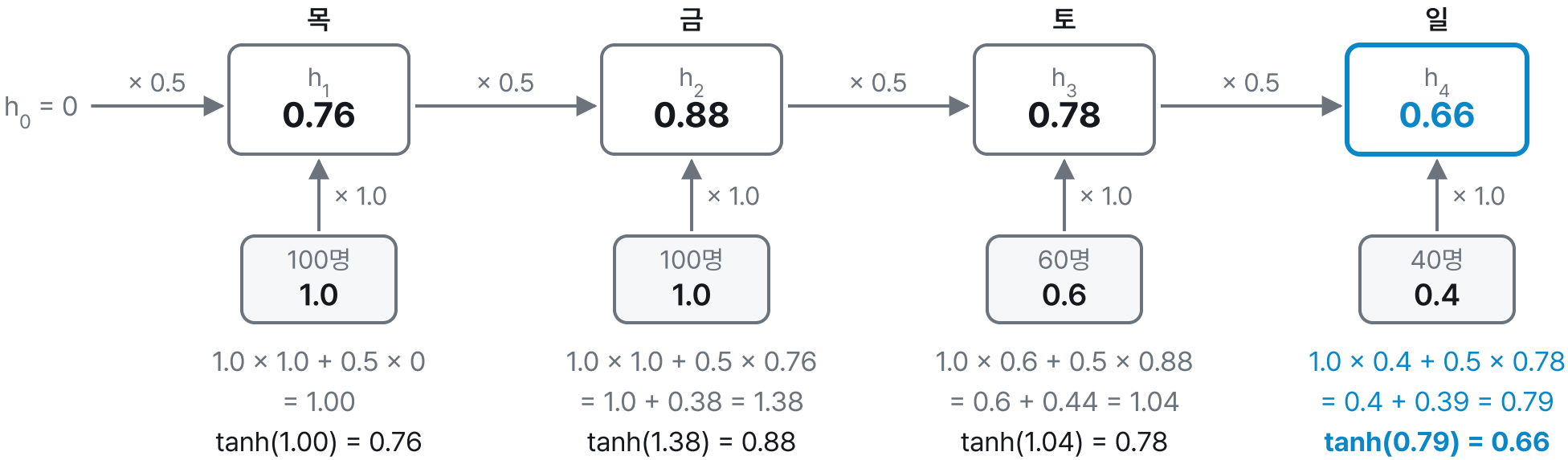


일요일: 1.0 × 0.4 + 0.5 × 0.78 = 0.79 이고 은닉 상태는 0.66 이다

1.0 × 0.4 + 0.5 × 0.78 = 0.4 + 0.39 = 0.79. 더한 값 0.79 가운데 0.39 가 토요일의 은닉 상태에서 왔다.

$$h_t = \tanh(w_x \times x_t + w_h \times h_{t-1})$$

$w_x = 1.0 \cdot w_h = 0.5$ ($W_{xh} \cdot W_{hh}$ 가 수 하나일 때의 이름) $\cdot h_0 = 0$

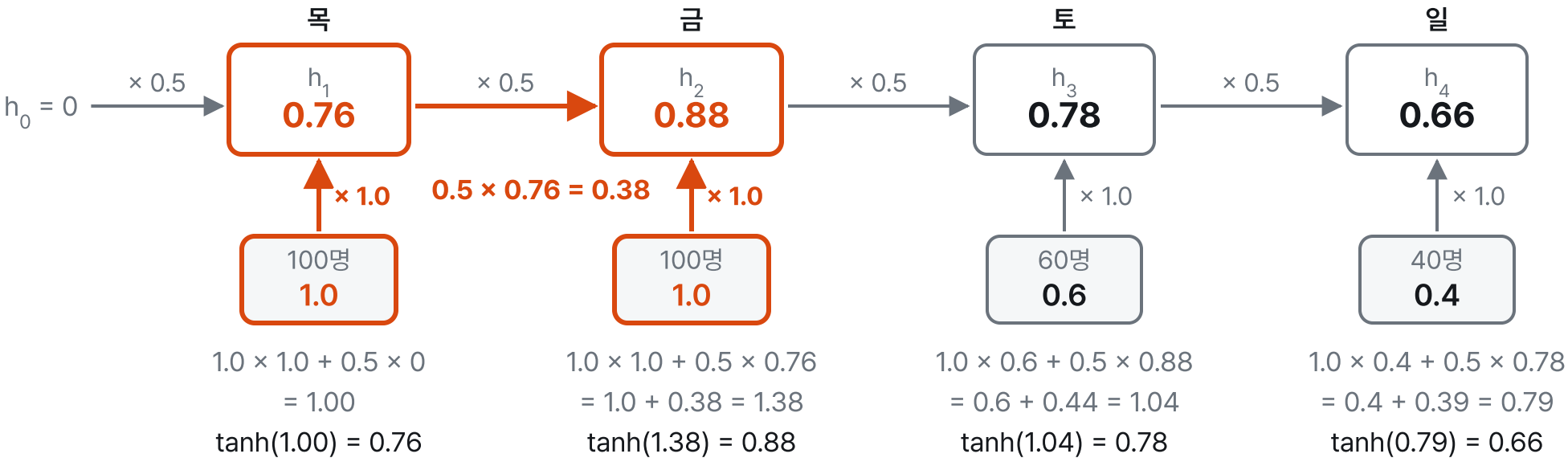


목요일과 금요일은 입력이 같아도 은닉 상태가 다르다: 앞 날의 계산 결과가 반영된다

목 · 금은 입력이 1.0 으로 같다. 금요일에는 목요일 은닉 상태의 절반 0.38 이 더해져 0.88 이 된다. 은닉 상태에는 앞 날들의 계산 결과가 반영된다.

$$h_t = \tanh(w_x \times x_t + w_h \times h_{t-1})$$

$w_x = 1.0 \cdot w_h = 0.5$ ($W_{xh} \cdot W_{hh}$ 가 수 하나일 때의 이름) $\cdot h_0 = 0$



torch.nn.RNN 에 같은 가중치를 지정하면 직접 계산한 값과 같은 은닉 상태가 나온다

```
import torch

rnn = torch.nn.RNN(input_size=1, hidden_size=1, bias=False, batch_first=True)
with torch.no_grad():
    rnn.weight_ih_l0.fill_(1.0)    # 지금 입력에 곱하는 가중치
    rnn.weight_hh_l0.fill_(0.5)    # 앞 은닉 상태에 곱하는 가중치

x = torch.tensor([[[1.0], [1.0], [0.6], [0.4]]])    # 목 · 금 · 토 · 일
out, h_last = rnn(x)
print(out.detach().flatten())      # 날마다의 은닉 상태
print(round(h_last.item(), 4))     # 마지막 날의 은닉 상태
```

출력

```
tensor([0.7616, 0.8811, 0.7781, 0.6579])
0.6579
```

출력 첫 행의 네 값이 날마다의 은닉 상태다. 직접 계산한 0.76 · 0.88 · 0.78 · 0.66 과 같다.

PyTorch 의 가중치 이름은 식의 $w_x \cdot w_h$ 와 짝이 맞고, 출력을 소수 둘째 자리로 반올림하면 직접 계산한 값과 같다

이름 대응

식의 기호	PyTorch 의 이름	이 예의 값
w_x (W_{xh})	`weight_ih_l0`	1.0
w_h (W_{hh})	`weight_hh_l0`	0.5
W_{hy}	없음. `torch.nn.Linear` 를 따로 이어 쓴다	
편향	`bias=False` 로 껐다	0

은닉 상태 대조

날	직접 계산	PyTorch
목	0.76	0.7616
금	0.88	0.8811
토	0.78	0.7781
일	0.66	0.6579

- 입력 모양은 (배치 1, 날 4, 값 1)이다.
- `out` 은 날마다의 은닉 상태, `h_last` 는 마지막 날의 은닉 상태다.
- `torch.nn.RNN` 의 활성화 함수 기본값은 tanh 다.

정리: 은닉 상태에는 그날의 입력과 앞 날들의 계산 결과가 함께 반영된다

1 날마다 같은 계산

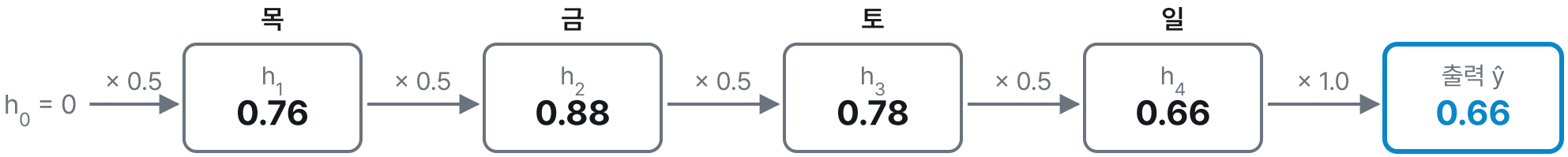
입력 × 1.0 에 앞 은닉 상태 × 0.5 를 더해 tanh 를 적용한다

2 앞 날이 반영된다

입력이 같은 목 · 금의 은닉 상태가 0.76 · 0.88 로 다르다. PyTorch 출력과 같다

3 예측은 출력으로

은닉 상태는 손님 수가 아니라 중간값이다. 출력 가중치 w_y(이 예에서는 1.0)를 곱해 예측을 구한다



출력 $\hat{y} = w_y \times h_4 = 1.0 \times 0.66 = 0.66$, 100 을 곱해 손님 수로 바꾸면 월요일 예측은 66명이다

실제는 100명이다. 가중치를 아직 학습하지 않았다(「5. RNN 의 학습과 한계」에서 고친다)

다음, 4. RNN 의 입출력 형태와 쓰임: 여기서는 마지막 날의 출력 하나만 썼다. 출력을 쓰는 시점에 따라 형태가 나뉜다.

4

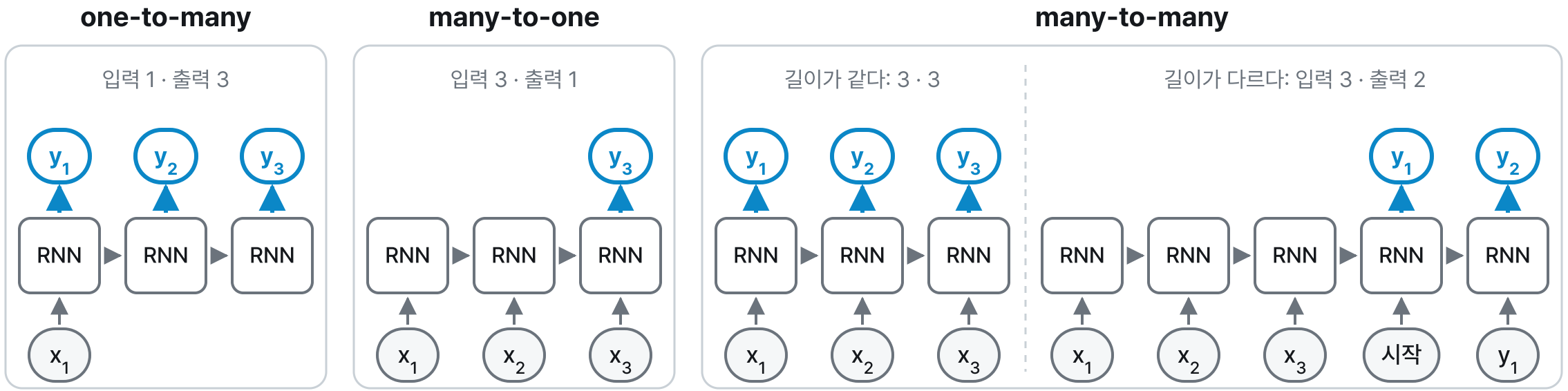
RNN 의 입출력 형태와 쓰임

입력과 출력의 수가 달라지면 같은 RNN 을 어떻게 쓰나?

입력과 출력의 수로 나눈 세 형태와 그 쓰임

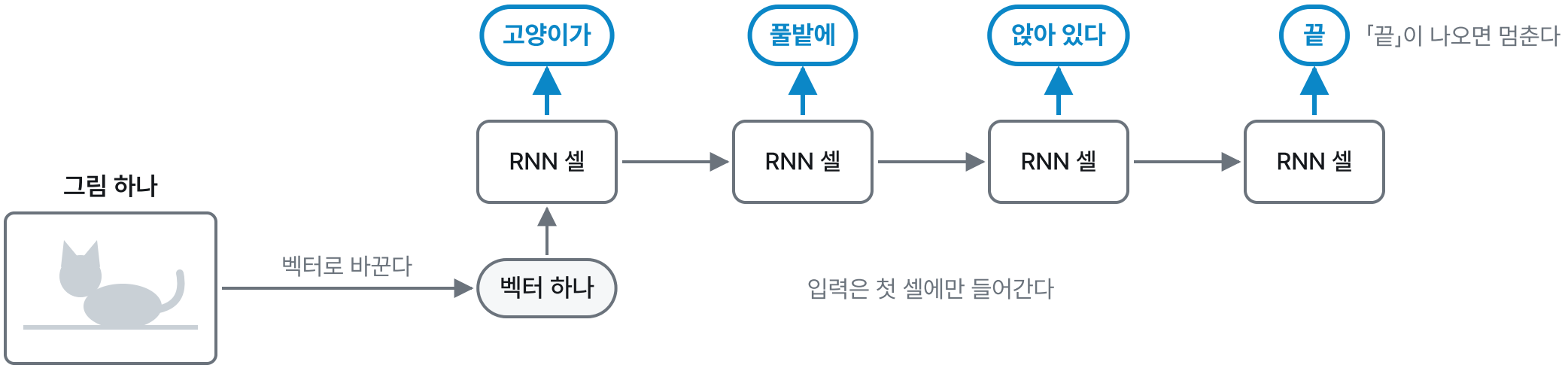
one-to-many(입력 하나에 출력 여럿) · many-to-one(입력 여럿에 출력 하나) · many-to-many(입력 여럿에 출력 여럿)

RNN 은 입력과 출력을 두는 시점에 따라 세 형태로 쓰고, many-to-many 는 길이에 따라 둘로 나뉜다



정해진 종류가 넷 있는 것이 아니다. 같은 RNN 셀(그림의 상자)을 두고 어느 시점에 입력을 받고 어느 시점의 출력을 쓰느냐가 다르다. **파랑**이 쓰는 출력이다. many-to-many 의 「길이가 다르다」 그림에서 「시작」은 출력을 내기 시작하라는 기호이고, 그 뒤에는 앞에 낸 출력 y_1 을 다음 입력으로 쓴다.

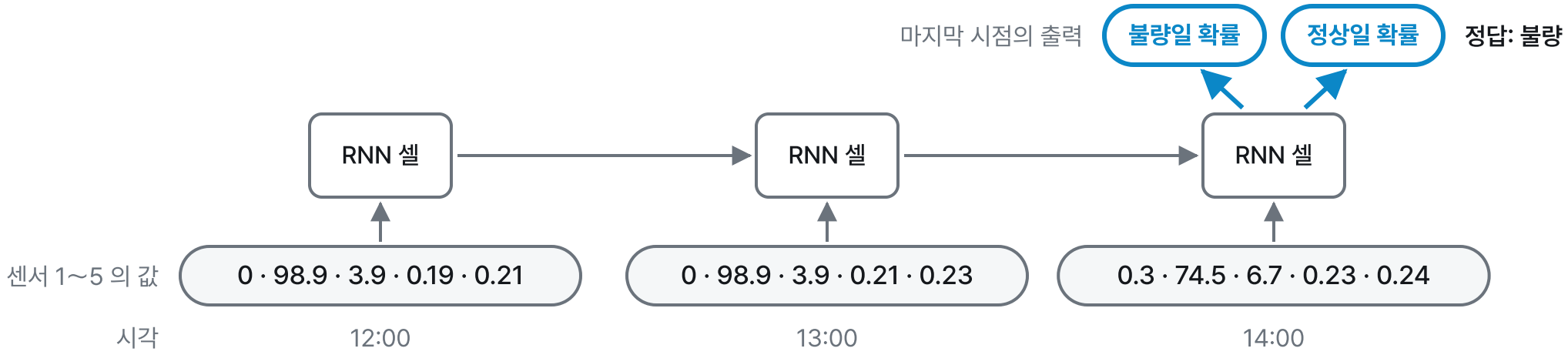
one-to-many: 입력 하나를 받아 출력을 차례로 여러 개 낸다



이미지 캡셔닝(image captioning)은 그림 하나를 받아 설명 문장의 낱말을 하나씩 낸다. 그림은 벡터(수 여러 개) 하나로 바꿔 입력한다. 출력은 「끝」 기호가 나올 때까지 내고, 최대 길이만 미리 지정한다.

many-to-one: 여러 시점을 받아 마지막 시점의 출력 하나만 쓴다

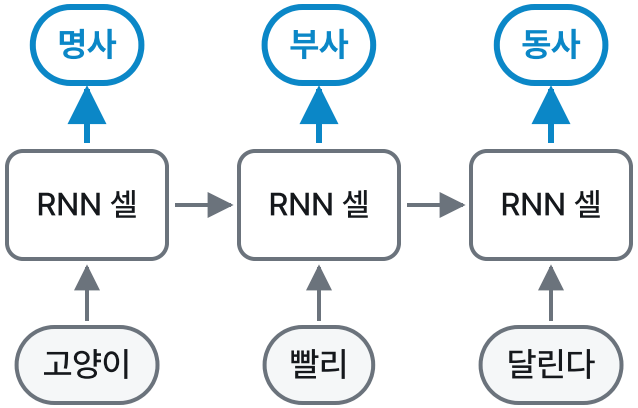
세 시점의 센서 값을 차례로 입력하고 마지막 셀의 출력으로 정상 · 불량을 분류한다. 분류라서 출력에 소프트맥스를 쓴다.



같은 형태: 감정 분석(문장의 낱말들 → 긍정 · 부정), 가게(지난 7일 → 내일 손님 수. 수를 예측하므로 소프트맥스를 쓰지 않는다)

many-to-many: 길이가 같으면 시점마다 출력을 쓰고, 길이가 다르면 Seq2Seq 형태로 쓴다

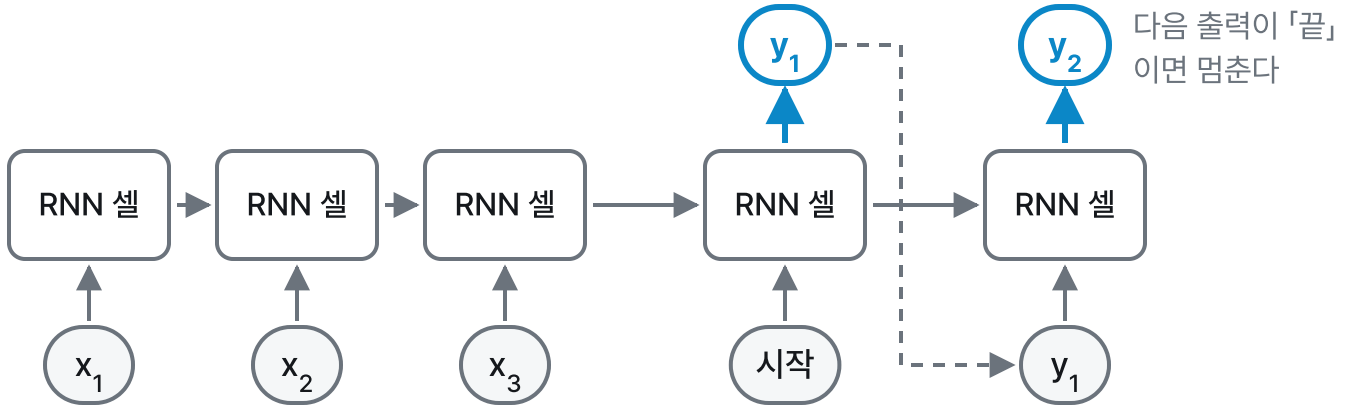
길이가 같다: 시점마다 출력을 쓴다



품사 태깅: 입력 3 → 출력 3

길이가 같고 시점마다 입력과 출력이 대응하는 예는 품사 태깅(낱말마다 명사 · 동사 같은 품사를 지정하는 일)이다. 가게로는 날마다 「평소와 다른 날인가」를 표시하는 일이다.

길이가 달라도 된다: Seq2Seq



번역: 입력 3 → 출력 2

입력과 출력의 길이가 달라도 되는 형태가 **Seq2Seq(sequence-to-sequence)**다. 예는 번역이고, 가게로는 과거 28일을 입력해 다음 7일을 예측하는 일이다. 구조는 다음 자료 「LSTM」의 「3. Seq2Seq 와 그 뒤」에 있다.

정리: 같은 RNN 셀을 두고 입력과 출력을 어느 시점에 두느냐만 바뀐다

형태	입력 · 출력	쓰임	가게로 바꾸면
one-to-many	입력 하나, 출력 여럿	이미지 캡셔닝	없음
many-to-one	입력 여럿, 마지막 시점의 출력 하나	센서 값으로 정상 · 불량 분류, 감정 분석	지난 7일 → 내일 손님 수
many-to-many 길이가 같다	시점마다 입력 하나에 출력 하나	품사 태깅	날마다 「평소와 다른 날인가」 표시
many-to-many 길이가 다르다(Seq2Seq)	입력을 모두 받은 뒤 출력을 하나씩	번역	과거 28일 → 다음 7일 손님 수

딥러닝 시계열의 예측은 many-to-one(내일)과 many-to-many(다음 7일)이다.

> **다음, 5. RNN 의 학습과 한계:** RNN 의 가중치를 학습하는 세 단계와, 먼 시점의 값이 줄어드는 문제.

5

RNN 의 학습과 한계

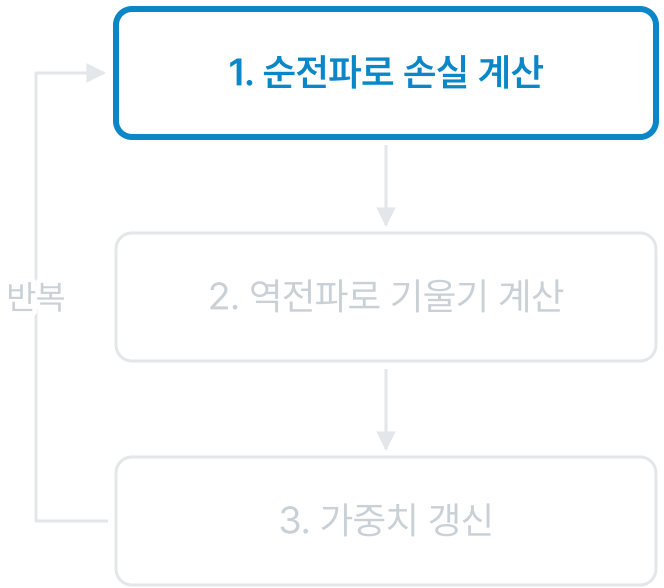
RNN 은 가중치를 어떻게 고치고, 먼 날의 값은 왜 덜 반영되나?

학습의 세 단계, 장기 의존성 문제, 기울기 소실

1단계 순전파: 지금 가중치로 계산한 월요일 예측은 66명이고 손실은 0.12 다

출력 $\hat{y} = w_y \times h_4$ · 맞힐 값 $y = 1.0$(다음 날 월요일 100명) · 손실 $= (\hat{y} - y)^2$

「딥러닝 개요」와 같은 세 단계로 학습한다. 「3. RNN 을 직접 계산하기」의 끝에서 구한 출력 0.66(66명)과 실제 100명의 차이로 손실을 계산한다. 가중치는 아직 학습하지 않은 값이다.



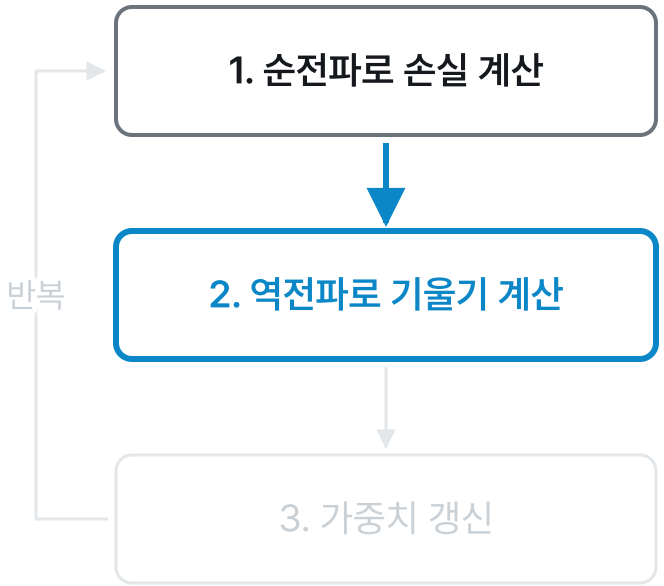
가중치가 w_x 1.0 · w_h 0.5 · w_y 1.0 일 때 손실은 얼마인가?

ŷ = 1.0 × 0.66 = 0.66(66명), 손실 = (0.66 − 1.0)² = 0.12

2단계 역전파: 손실에서 시점을 거슬러 가며 가중치마다 기울기를 구한다

출력 $\hat{y} = w_y \times h_4$ · 맞힐 값 $y = 1.0$(다음 날 월요일 100명) · 손실 $= (\hat{y} - y)^2$

w_y 의 기울기는 「딥러닝 개요」의 연쇄 법칙 그대로 직접 계산할 수 있다. 셋 다 음수라 가중치를 키우면 손실이 준다. RNN 의 역전파는 시점을 거슬러 가므로 시간 역전파(backpropagation through time, BPTT)라 부른다.



가중치가 w_x 1.0 · w_h 0.5 · w_y 1.0 일 때 손실은 얼마인가?

$\hat{y}$ = 1.0 × 0.66 = 0.66(66명), 손실 = (0.66 − 1.0)² = 0.12

가중치를 조금 키우면 손실은 얼마나 바뀌는가?

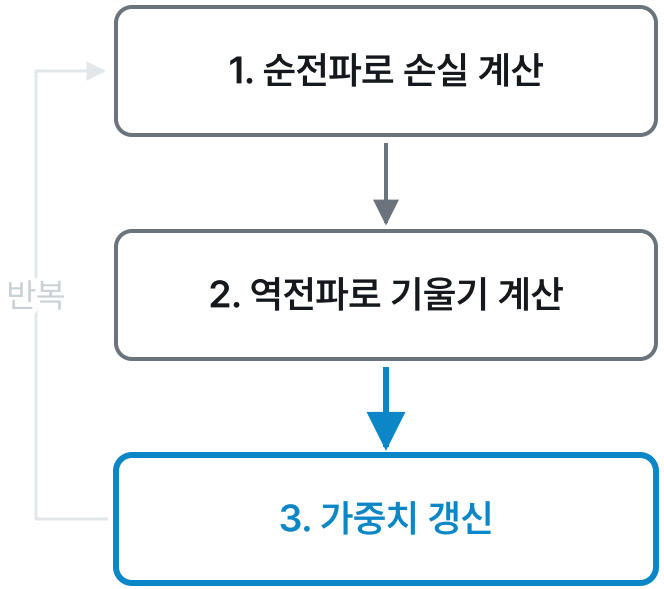
w_y 의 기울기 = 2 × ($\hat{y}$ − y) × h_4 = 2 × (0.66 − 1.0) × 0.66 = −0.45

w_x 의 기울기 −0.21 · w_h 의 기울기 −0.38 (시점을 거슬러 가야 해서 PyTorch 자동 미분으로 구한 값)

3단계 갱신: 기울기의 반대쪽으로 학습률 0.1 만큼 가중치를 옮긴다

출력 $\hat{y} = w_y \times h_4$ · 맞힐 값 $y = 1.0$(다음 날 월요일 100명) · 손실 $= (\hat{y} - y)^2$

가중치 묶음은 하나라 한 번 고친다(날마다 따로 고치지 않는다). 기울기가 음수이므로 그 반대쪽, 곧 가중치를 키우는 쪽으로 옮긴다.



가중치가 w_x 1.0 · w_h 0.5 · w_y 1.0 일 때 손실은 얼마인가?

$\hat{y}$ = 1.0 × 0.66 = 0.66(66명), 손실 = (0.66 − 1.0)² = 0.12

가중치를 조금 키우면 손실은 얼마나 바뀌는가?

w_y 의 기울기 = 2 × ($\hat{y}$ − y) × h_4 = 2 × (0.66 − 1.0) × 0.66 = −0.45

w_x 의 기울기 −0.21 · w_h 의 기울기 −0.38 (시점을 거슬러 가야 해서 PyTorch 자동 미분으로 구한 값)

가중치를 어느 쪽으로 얼마나 옮기는가?

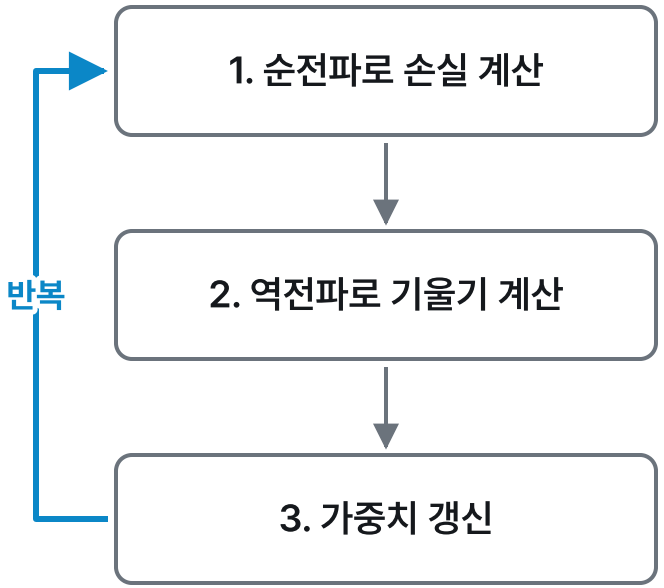
새 가중치 = 가중치 − 0.1 × 기울기

w_x 1.0 → 1.021 · w_h 0.5 → 0.538 · w_y 1.0 → 1.045

세 단계를 한 번 거치면 예측이 66명에서 72명이 되고 손실은 0.12 에서 0.08 로 준다

출력 $\hat{y} = w_y \times h_4$ · 맞힐 값 $y = 1.0$(다음 날 월요일 100명) · 손실 $= (\hat{y} - y)^2$

새 가중치로 목 · 금 · 토 · 일을 처음부터 다시 계산하면 예측이 66명에서 72명으로 실제 100명에 가까워진다. 이 세 단계를 반복한다.



가중치가 w_x 1.0 · w_h 0.5 · w_y 1.0 일 때 손실은 얼마인가?

$\hat{y}$ = 1.0 × 0.66 = 0.66(66명), 손실 = (0.66 − 1.0)² = 0.12

새 가중치로 나흘을 처음부터 다시 계산하면 $\hat{y}$ = 0.72(72명), 손실 0.08

가중치를 조금 키우면 손실은 얼마나 바뀌는가?

w_y 의 기울기 = 2 × ($\hat{y}$ − y) × h_4 = 2 × (0.66 − 1.0) × 0.66 = −0.45

w_x 의 기울기 −0.21 · w_h 의 기울기 −0.38 (시점을 거슬러 가야 해서 PyTorch 자동 미분으로 구한 값)

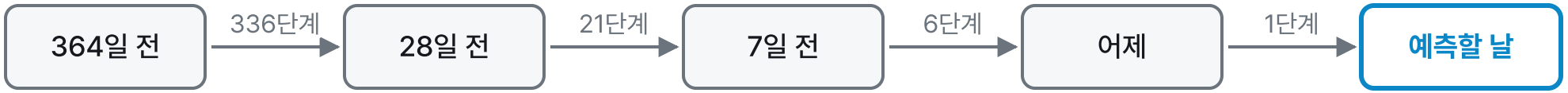
가중치를 어느 쪽으로 얼마나 옮기는가?

새 가중치 = 가중치 − 0.1 × 기울기

w_x 1.0 → 1.021 · w_h 0.5 → 0.538 · w_y 1.0 → 1.045

한 단계마다 0.9배가 되면 7일 전 값은 0.48배, 28일 전 값은 0.05배만 반영된다

어느 날의 은닉 상태가 조금 달라졌을 때 다음 날의 은닉 상태가 달라지는 크기가 그 **0.9배**라고 하자(딥러닝 시계열 2편의 수).



	364일 전	28일 전	7일 전	어제
거치는 단계 수	364	28	7	1
반영되는 크기	0 에 가까움	**0.05**	0.48	0.90

7일 전의 값은 0.9 를 7번 곱한 0.48배만, 28일 전의 값은 **0.05배**만 지금 은닉 상태에 반영된다. 먼 날의 입력은 여러 단계를 거쳐 전달되고, 반영되는 크기는 단계 수만큼 거듭 줄어든다. 이것이 **장기 의존성 문제(long-term dependency problem)**다.

> 앞에서 직접 계산한 RNN 이 한 단계에 실제로 곱하는 수는 0.11~0.28 이다(뒤의 역전파 그림에서 하나씩 구한다).

역전파는 한 단계 거슬러 갈 때마다 w_h × (tanh 의 미분)을 곱하고, 이 미분은 1 이하다

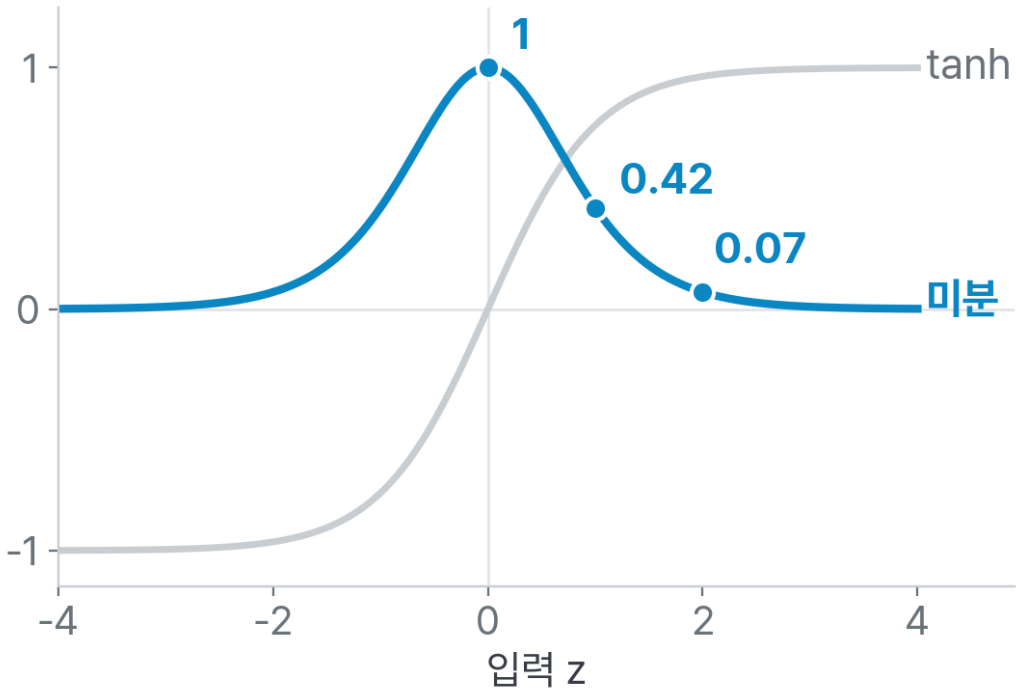
「딥러닝 개요」의 역전파는 층을 거슬러 갔고, RNN 의 역전파는 시점을 거슬러 간다. 한 단계 거슬러 갈 때 곱하는 수는 다음과 같다.

$$\frac{\partial h_t}{\partial h_{t-1}} = w_h \times (1 - h_t^2)$$

$\partial h_t / \partial h_{t-1}$: 앞 은닉 상태가 조금 달라질 때 이번 은닉 상태가 달라지는 크기(앞 장의 0.9배에 해당하는 수)
w_h: 앞 은닉 상태에 곱하는 가중치
$1 - h_t^2$: tanh 의 미분(0 초과 1 이하, 입력이 0 일 때만 1)

확인: tanh(1) = 0.76 이고 1 − 0.76² = 0.42, 그래프의 점과 같다.

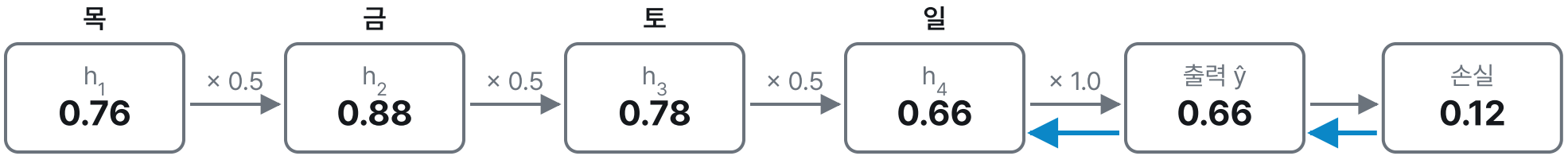
> 0 과 1 사이인 것은 tanh 의 미분뿐이고, 가중치 w_h 가 함께 곱해진다.



회색은 tanh, 파랑은 tanh 의 미분이다. 점에 적은 수는 입력 0 · 1 · 2 일 때의 미분이다.

역전파는 손실에서 출발해 일요일의 은닉 상태부터 앞 날로 거슬러 간다

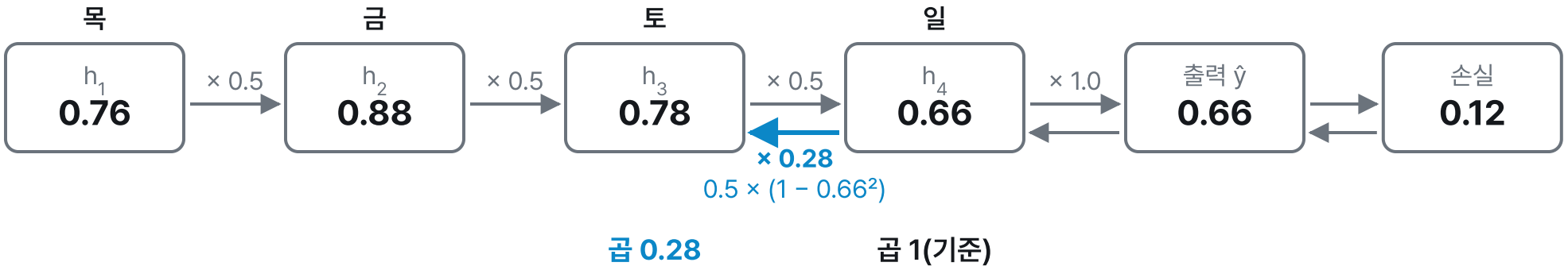
앞 장의 수 「0.5 × (1 − 그날 은닉 상태²)」를 날마다 곱해 간다. 그림 아래의 「곱」은 그날의 은닉 상태가 조금 달라질 때 일요일의 은닉 상태가 그 몇 배만큼 달라지는지를 나타낸다. 일요일은 기준이라 1 이다. 0.9배는 간단한 예였고, 여기서는 직접 계산한 RNN 이 실제로 곱하는 수를 쓴다.



곱 1(기준)

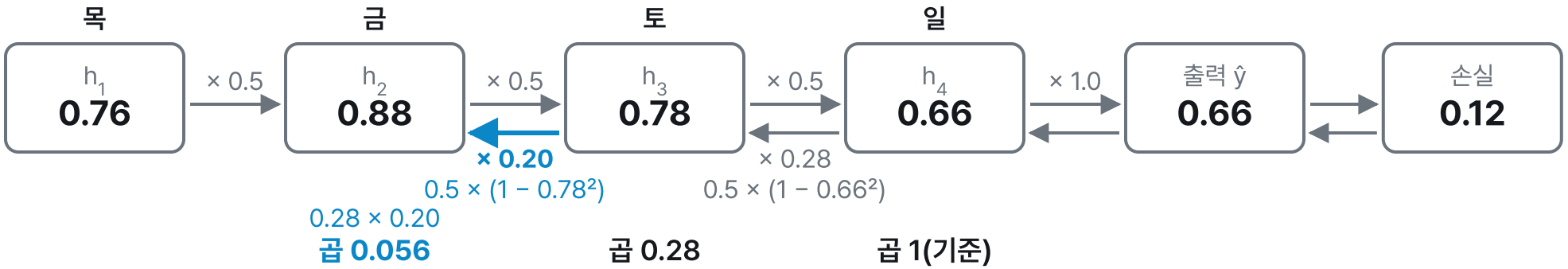
한 단계 거슬러 간 토요일에는 0.28 이 곱해진다

일요일의 계산을 거슬러 갈 때 0.5 × (1 − 0.66²) = 0.28 을 곱한다. 0.5 는 w_h 이고, 1 − 0.66² 은 일요일 은닉 상태 0.66 으로 구한 tanh 의 미분이다. 곱 0.28 은 토요일 은닉 상태가 조금 달라지면 일요일 은닉 상태가 그 0.28배만큼 달라진다는 뜻이다.



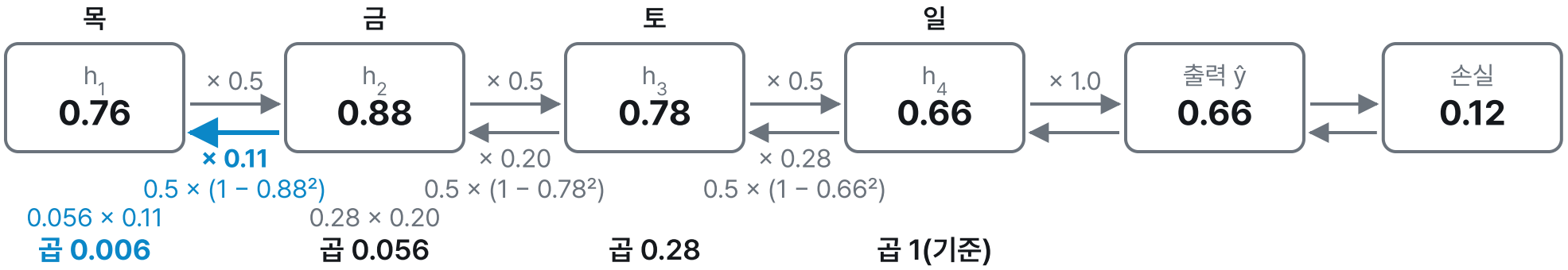
두 단계 거슬러 간 금요일에는 0.28 × 0.20 = 0.056 이 곱해진다

토요일의 계산을 거슬러 갈 때 0.5 × (1 − 0.78²) = 0.20 을 한 번 더 곱한다. 금요일에 곱해진 수는 0.28 × 0.20 = 0.056 이다. 금요일 은닉 상태가 조금 달라지면 일요일 은닉 상태는 그 0.056배만큼 달라진다.



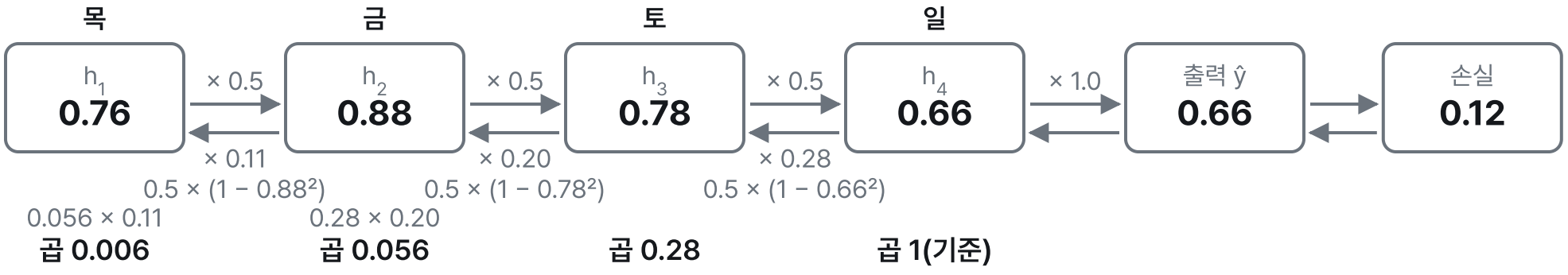
세 단계 거슬러 간 목요일에는 0.006 이 곱해진다: 사흘 만에 1,000분의 6 이다

금요일의 계산을 거슬러 갈 때 0.5 × (1 − 0.88²) = 0.5 × 0.2256 = 0.11 을 곱한다. 0.056 × 0.11 = 0.006 이므로 사흘 만에 1,000분의 6 이다.



w_x 의 기울기 −0.21 가운데 73% 가 일요일 항이고 목요일 항은 1% 다

w_x 는 네 날 모두에서 쓰였으므로 w_x 의 기울기는 날마다의 항(기울기 항) 넷을 더한 값이다. 그날의 항은 아래 식의 네 수를 곱한 값이고, 첫째 수 $2 \times (\hat{y} - y) \times w_y = 2 \times (0.66 - 1.0) \times 1.0 = -0.68$ 은 네 날 모두 같다. 먼 날일수록 곱이 작아 항도 작다.



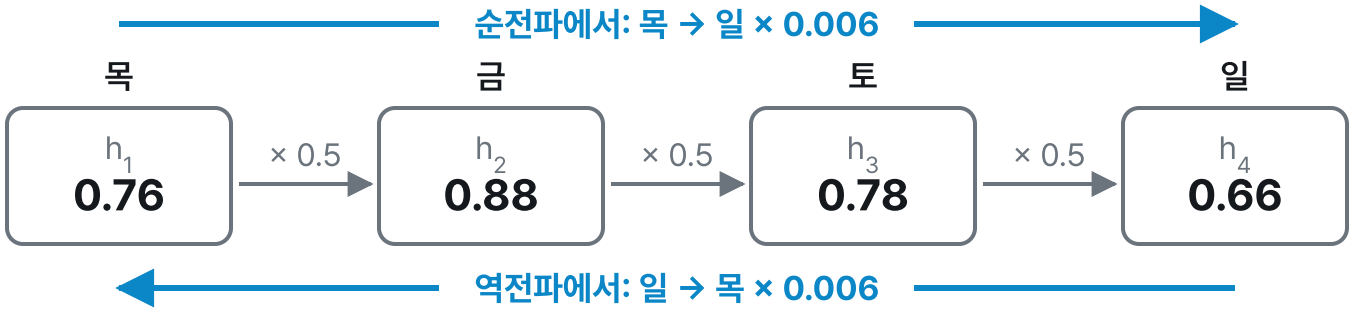
그날의 항	=	2 × (ŷ − y) × w_y	×	곱	×	(1 − 그날 은닉 상태²)	×	그날 입력		
일요일 항	=	−0.68	×	**1**	×	(1 − 0.66²)	×	0.4	=	**−0.154**
토요일 항	=	−0.68	×	**0.28**	×	(1 − 0.78²)	×	0.6	=	**−0.045**
금요일 항	=	−0.68	×	**0.056**	×	(1 − 0.88²)	×	1.0	=	**−0.009**
목요일 항	=	−0.68	×	**0.006**	×	(1 − 0.76²)	×	1.0	=	**−0.002**

73% = 0.154 ÷ 0.210

1% = 0.002 ÷ 0.210

넷의 합 = −0.154 − 0.045 − 0.009 − 0.002 = −0.210 (2단계 역전파에서 자동 미분으로 구한 기울기 −0.21 과 같다)

장기 의존성 문제와 기울기 소실은 같은 수가 곱해지는 현상을 순전파와 역전파에서 본 것이다



기울기 전체가 0 이 되는 것은 아니다. 가까운 날의 항은 남아 학습은 되고, 가까운 날의 값만 반영하도록 학습된다.

	순전파에서 보면	역전파에서 보면
이름	장기 의존성 문제	기울기 소실(vanishing gradient)
무슨 일인가	먼 날의 은닉 상태가 달라져도 지금 은닉 상태는 조금만 달라진다	먼 날로 거슬러 간 기울기 항이 0 에 가까워진다
나흘 예	목요일 은닉 상태가 달라진 크기의 0.006배만 일요일 은닉 상태에 반영된다	일요일에서 목요일로 거슬러 간 기울기에 0.006 이 곱해진다
28일이면	가게 28일을 입력하면 평일이 이어지는 날에 은닉 상태가 0.895 까지 올라, 한 단계에 곱하는 수가 0.10~0.28 이다(은닉 상태가 0.88 까지 오른 나흘 예는 0.11~0.28). 이 수를 27번 곱하면 2×10^{-24} 이 된다(파이썬으로 계산). 두 방향에서 곱해지는 수가 같다.	

곱하는 수가 1 보다 크면 곱은 반대로 커진다. 1.1 을 28번 곱하면 14배다. 이것이 기울기 폭발(exploding gradient)이고, 기울기가 정해 둔 크기를 넘으면 그 크기로 줄이는 기울기 자르기(gradient clipping)로 대처한다.

정리: RNN 은 세 단계로 학습하고, 먼 날로 갈수록 곱이 줄어 먼 날의 값이 덜 반영된다

1 세 단계를 반복한다

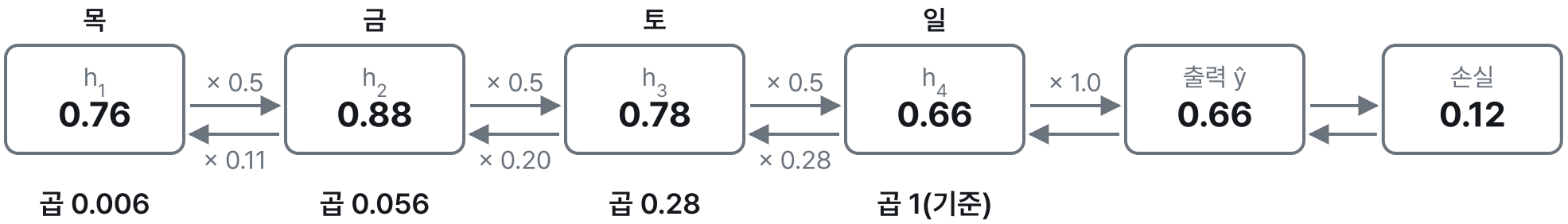
순전파 → 역전파 → 갱신을 한 번 거치면
손실이 0.12 에서 0.08 로 준다

2 거슬러 갈수록 곱이 준다

한 단계마다 w_h × (tanh 의 미분)을 곱해
사흘 만에 0.006 이 된다

3 먼 날은 덜 반영된다

그래서 먼 날의 값을 반영하도록 가중치가
고쳐지기 어렵다



다음 자료, 「LSTM」: 한 단계에 곱하는 수가 1 에 가까운 전달 경로를 하나 더 두는 LSTM 의 구조와 계산 예, 그리고 Seq2Seq 를 다룬다.